

Sistemas Expertos

Julio Garcia Ortiz



1. Documentación.
2. Preguntitas

2
3

1. Documentación.

a. Reglas

Definimos la base del conocimiento mediante la estructura de datos, esto nos permite fácil escalabilidad y mantenimiento de las reglas, según mi parecer bastante similares a lo visto con prolog.

```
1 # Reglas del sistema experto
2 REGLAS = [
3     {"id": "R1", "if": ["adjunto_sospechoso", "macro_activa"], "then": "phishing_probable", "op": "AND"},
4     {"id": "R2", "if": ["antivirus_alerta", "ejecucion_binario"], "then": "malware_probable", "op": "AND"},
5     {"id": "R3", "if": ["intentos_login_altos", "pais_inusual"], "then": "compromiso_cuenta_probable", "op": "AND"},
6     {"id": "R4", "if": ["acceso_fuera_horario", "multiples_hosts"], "then": "movimiento_lateral_probable", "op": "AND"},
7     {"id": "R5", "if": ["descarga_masiva", "acceso_fuera_horario"], "then": "exfiltracion_probable", "op": "AND"},
8     {"id": "R6", "if": ["malware_probable", "movimiento_lateral_probable"], "then": "incidente_critico", "op": "AND"},
9     {"id": "R7", "if": ["compromiso_cuenta_probable", "exfiltracion_probable"], "then": "incidente_critico", "op": "AND"},
10    {"id": "R8", "if": ["phishing_probable", "compromiso_cuenta_probable"], "then": "acceso_inicial_exitoso", "op": "AND"},
11    {"id": "R9", "if": ["acceso_inicial_exitoso", "malware_probable"], "then": "ataque_multietapa", "op": "AND"},
12    {"id": "R10", "if": ["ataque_multietapa"], "then": "respuesta_prioritaria", "op": "OR"},
13    {"id": "R11", "if": ["incidente_critico"], "then": "respuesta_prioritaria", "op": "OR"},
14 ]
15
16 CAMPOS_OBLIGATORIOS = [
17     "id_evento", "usuario", "ip_reputacion", "pais_inusual", "intentos_login",
18     "adjunto_sospechoso", "macro_activa", "ejecucion_binario",
19     "acceso_fuera_horario", "volumen_descarga_mb", "multiples_hosts",
20     "antivirus_alerta"
21 ]
```

b. Carga de datos

Definimos la ruta del archivo que usaremos, en este caso usando Import OS para obtener la ruta del archivo llamado csv y transformamos los valores brutos del csv en hechos simples para poder trabajar con ellos mediante las reglas.

```
1 def cargar_csv():
2     # Obtener la ruta del directorio donde está el script
3     base_path = os.path.dirname(os.path.abspath(__file__))
4     ruta = os.path.join(base_path, ARCHIVO_CSV)
5
6     if not os.path.exists(ruta):
7         raise FileNotFoundError(f"No se encuentra {ARCHIVO_CSV} en {base_path}")
8
9     casos = []
10    with open(ruta, newline="", encoding="utf-8") as f:
11        lector = csv.DictReader(f)
12        for fila in lector:
13            hechos = set()
14            if es_si(fila["pais_inusual"]): hechos.add("pais_inusual")
15            if es_si(fila["adjunto_sospechoso"]): hechos.add("adjunto_sospechoso")
16            if es_si(fila["macro_activa"]): hechos.add("macro_activa")
17            if es_si(fila["ejecucion_binario"]): hechos.add("ejecucion_binario")
18            if es_si(fila["acceso_fuera_horario"]): hechos.add("acceso_fuera_horario")
19            if es_si(fila["multiples_hosts"]): hechos.add("multiples_hosts")
20            if es_si(fila["antivirus_alerta"]): hechos.add("antivirus_alerta")
21            if int(fila["intentos_login"]) > 5: hechos.add("intentos_login_altos")
22            if float(fila["volumen_descarga_mb"]) > 1000: hechos.add("descarga_masiva")
23
24        casos.append({"id": fila["id_evento"], "usuario": fila["usuario"], "hechos": hechos})
25    return casos
```

c. Forward chaining

La inferencia hacia delante va evaluando de forma iterativa todas las reglas aplicables maximizando la detección de riesgos a partir de los hechos iniciales.

```
1 def forward_chaining(hechos_iniciales):
2     hechos = set(hechos_iniciales)
3     inferidos, usadas = [], []
4     cambio = True
5     while cambio:
6         cambio = False
7         for regla in REGLAS:
8             if regla["then"] not in hechos:
9                 cond = all(p in hechos for p in regla["if"]) if regla["op"] == "AND" else any(p in hechos for p in regla["if"])
10                if cond:
11                    hechos.add(regla["then"])
12                    inferidos.append(regla["then"])
13                    usadas.append(regla["id"])
14                    cambio = True
15     return hechos, inferidos, usadas
```

d. Backward chaining

A diferencia de la inferencia hacia delante, esta realiza búsquedas concretas dirigidas a verificar hipótesis críticas del sistema, optimizando su funcionamiento ya que buscamos casos específicos.

```
1 def backward_chaining(obj, hechos, visitados=None):
2     if visitados is None: visitados = set()
3     if obj in hechos: return True
4     if obj in visitados: return False
5     visitados.add(obj)
6     for regla in REGLAS:
7         if regla["then"] == obj:
8             if all(backward_chaining(p, hechos, visitados) for p in regla["if"]) if regla["op"] == "AND" else any(backward_chaining(p, hechos, visitados) for p in regla["if"]):
9                 return True
10    return False
```

2. Preguntitas

1. ¿Cómo se ha representado la base de conocimientos?

Principalmente con el diccionario de Python, aquí representamos cada regla con un operador lógico, premisas y conclusiones.

2. ¿Cómo se construye la base de hechos a partir del CSV?

En el CSV obtenemos todos los datos que trataremos, cada clave extrae valores y los convierte en hechos iniciales o derivados que se almacenan en un set.

3. ¿Cómo se ha implementado el forward chaining?

En un bucle while que recorre todas las reglas, si todas las premisas de una regla están en la mayoría de los hechos, se añade la conclusión y se repite el proceso hasta que no hay nuevas inferencias.

4. ¿Cómo se ha implementado el backward chaining?

Buscamos reglas en las cuales la conclusión sea su hipótesis y se intenta probar sus propias premisas de forma recursiva hasta llegar a los hechos conocidos o agotar las reglas.

5. ¿Qué ventajas presenta el encadenamiento hacia delante en este problema?

Desde el principio podemos hacer el análisis y permite detectar automáticamente todos los posibles riesgos asociados a un incidente de forma global.

6. ¿Qué ventajas presenta el encadenamiento hacia atrás?

Principalmente la velocidad para verificar conclusiones concretas, ya que busca el objetivo exacto que deseamos verificar por lo que es bastante óptimo.

7. ¿Qué diferencias se observan en coste, pasos o claridad del razonamiento?

El forward chaining es más completo pero potencialmente más costoso en términos de cómputo al explorar todas las reglas. El backward chaining es más eficiente para consultas puntuales pero requiere una estructura de búsqueda recursiva.

8. ¿Qué limitaciones tiene el sistema desarrollado?

El sistema es determinista y no gestiona incertidumbre ni datos históricos. Sus reglas son estáticas y no pueden adaptarse a nuevos patrones de ataques sin modificación manual del código.

9. ¿Cómo podría ampliarse para tratar incertidumbre o probabilidades?

Se podría integrar lógica difusa (fuzzy logic) o modelos probabilísticos (como redes bayesianas) que asignen un nivel de confianza a cada conclusión inferida, en lugar de un valor booleano simple.